Data Science Portfolio

Hafza Abdullahi Submission Date: 27/03/2026 –

Project Overview & Technology Reasoning

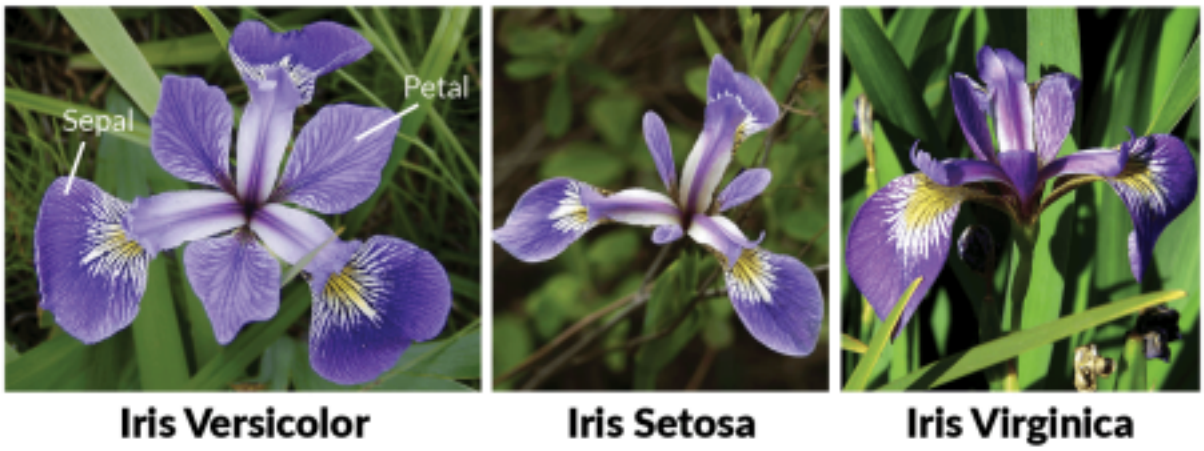
Objective: To develop a professional machine learning portfolio demonstrating proficiency across classical ML, Deep Learning, and Reinforcement Learning.

Technology choices (To be Updated)

1. Python
2. Jupyter Notebooks
3. Scikit-Learn, Seaborn

Dataset One - Iris Flower Dataset

For **chapters 2, 3, and 4** I focused on Classical Machine Learning comparison using the **Iris Dataset**, a dataset containing 3 classes: Setosa, Versicolor, and Virginica. The goal was to observe how different mathematical approaches handle the same data.



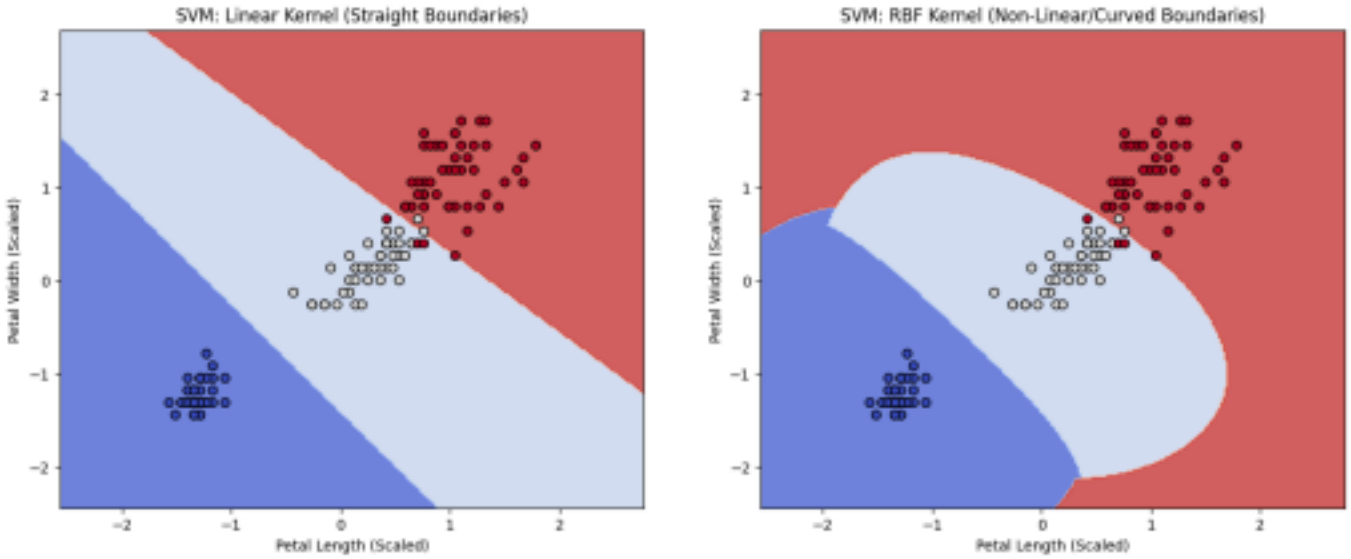
Iris Versicolor **Iris Setosa** **Iris Virginica**

Data Processing

- **Adjustment:** Split the dataset into a 70/30 train-test split using a fixed random state (42) for reproducibility.
- **Adjustment:** Applied StandardScaler to normalize the features
- **Practical Impact:** Distance-based algorithms (like KNN and K-Means) and margin-based algorithms (like SVM) perform significantly better and converge faster when features share the same scale

Algorithm 1: Support Vector Machines (SVM)

- **Implementation:** Developed two models to compare boundaries: a Linear SVM (this data can be separated by a straight line/plane) and an RBF SVM (using the 'Kernel Trick' for higher dimensions).
- **Analysis & Impact:** After scaling, the Linear SVM achieved an accuracy of 0.98, while the RBF SVM achieved a perfect accuracy of 1.00.
- **Visualization:** I created contour plots isolating Petal Length and Petal Width to visually contrast the straight decision boundaries of the linear kernel against the curved, non-linear boundaries of the RBF kernel.

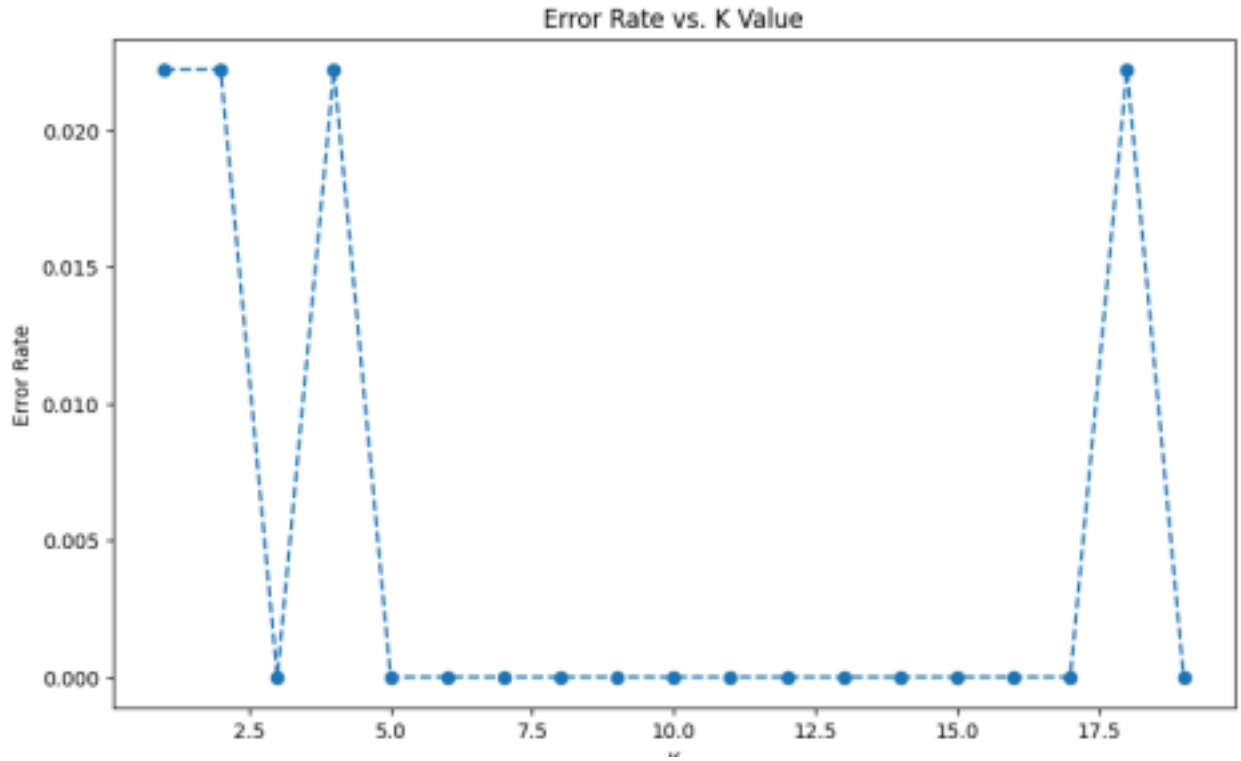


Algorithm 2: K-Nearest Neighbors (KNN)

The primary challenge with KNN is selecting the optimal K value. Plotting the accuracy and error rates against the K values to find the best k value for this dataset

Implementation: Built a KNN classifier and iterated through K values from 1 to 20.

Analysis & Impact: Plotted the accuracy and the Error Rate vs. K Value to systematically identify the optimal number of neighbors without underfitting or overfitting the model.



This graph proves that the KNN model is highly effective for this dataset. The transition from a 2.2% error at K=3 and back to a 0% error at K=5 shows how KNN is susceptible to local noise. A lower K value is better since overfitting to a single dataset is not ideal.

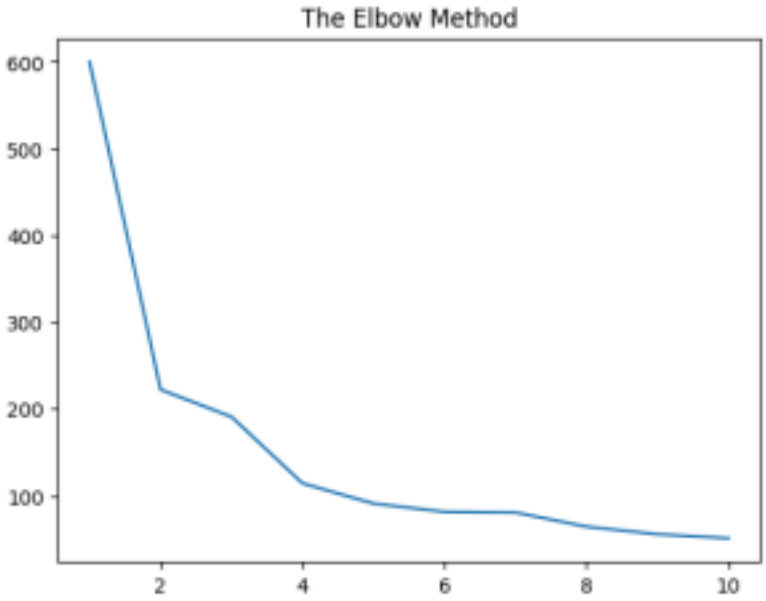
Algorithm 3: K-Means Clustering

Implementation:

Unlike SVM and KNN, which are Supervised Learning meaning the model is told the answers, I implemented K-Means as an Unsupervised Learning algorithm.

- I stripped away the l target species labels and gave the algorithm only the raw measurements (Petal/Sepal length and width).
- I ran the KMeans algorithm for K values ranging from 1 to 10. For each iteration, the model calculated the WCSS (Within-Cluster Sum of Squares), which measures the closeness of the clusters by calculating the distance between points and their respective centroids.

The primary goal of including K-Means is to demonstrate **Pattern Recognition** capabilities without human intervention such as labeled data the algorithm learns from



The Elbow Method

In this graph, a sharp decline can be observed which then flattens out

- Between 1 and 2: There is a massive drop in error. This means 1 cluster was a very **poor fit.**
- Between 2 and 3: There is another significant drop.
- **At k=3:** The bend or **Elbow** occurs and then flattens out. This shows the algorithm is naturally organizing the data into **three distinct groups**.

Chapters 5 & 6: Deep Learning — Siamese Pitch Accent Recognition

Dataset Two: Japanese Pitch Accent (OJAD) For this section, I moved beyond standard tabular data to unstructured audio data. The objective was to build a Siamese Neural Network to determine if two audio samples share the same Japanese pitch accent pattern.

Data Processing

- **Data Acquisition**: Developed a custom web scraper to extract thousands of audio files from the Online Japanese Accent Dictionary (OJAD).
- **Feature Engineering**: Converted raw audio into two distinct formats:
    - **1D Pitch Contours**: Extracted the fundamental frequency (F0) to track the "melody" or pitch changes of the word over time.
    - **2D Mel-Spectrograms**: Converted audio into frequency-over-time heatmaps to capture textural audio data.
- **Standardization**: Applied dynamic resampling and zero-padding to ensure all audio vectors were of identical length (100 frames) for consistent input into the neural network.

Algorithm 4: Siamese Neural Networks

- **Implementation**: I built a Siamese architecture consisting of two identical "twin" subnetworks that share weights. This allows the model to process two different inputs and calculate a similarity score.
- **Logic**: I implemented **Contrastive Loss** to optimize the network. Instead of telling the AI what the word is, this math tells the AI to pull samples with the same accent together and push different accents far apart in its memory.
- **Analysis & Impact**: The 2D CNN model successfully identified high-level patterns in the spectrograms, while the 1D model was more efficient at tracking specific pitch changes. This phase proved that Deep Learning can handle complex pattern recognition tasks that classical ML cannot.

Chapter 7: Reinforcement Learning — CartPole Balancing

Environment: Gymnasium CartPole-v1 The goal of this chapter was to transition from static learning to dynamic, agent-based learning. I trained an agent to balance a pole on a cart by applying horizontal forces, demonstrating the fundamental RL loop of Observation, Action, and Reward.

The Progression of Learning I implemented a three-stage progression to show how the model's logic improves as the mathematical approach changes.

1. The Baseline (Random Action)

- **Implementation**: The agent selected actions randomly (left or right) without any mathematical goal or logic.
- **Analysis & Impact**: The agent survived an average of only **25.2 steps**. This served as the control group, proving that the system is naturally unstable without a learned policy.

2. Deep Q-Network (DQN — Value-Based)

- **Implementation**: I introduced the **Bellman Equation** to help the agent estimate the "Quality" (Q) or value of future actions.
- **Logic**: The model improves by calculating the difference between its current prediction and the actual reward it received plus its estimate of future rewards. It tries to minimize this error over time.
- **Analysis & Impact**: While performance improved, the DQN agent was often unstable or "jittery" during training. It showed the limitations of value-based methods when trying to balance continuous physics.

3. Proximal Policy Optimization (PPO — Policy-Based)

- **Implementation**: I used PPO to optimize the AI's strategy directly using a "clipped" logic.
- **Logic**: PPO uses a **Clipped Objective** which essentially puts "guardrails" on the learning process. It prevents the AI from changing its strategy too drastically in a single update, which keeps the training smooth and stable.
- **Analysis & Impact**: The PPO agent achieved a perfect score of **500 steps**. This mathematical stability resulted in a much more reliable balancing policy than the previous models.

Final Comparative Analysis

This portfolio concludes by highlighting the relationship between data types and algorithm choice:

- **Classical ML (Iris)**: Most effective for structured data where clear boundaries can be drawn.
- **Deep Learning (Pitch Accent)**: Necessary for unstructured data like audio where the model must learn its own features.

- **Reinforcement Learning (CartPole)**: The only choice for dynamic environments where the agent must learn through trial and error.